Que la IA no invente

Listas cerradas, salidas verificables, set dorado y filtros a fondo. Cómo pasar de «se ve bien» a «lo puedo demostrar».

Jonathan Rodríguez
Construir herramientas con IA, sin ser programador

Antes de empezar

En la guía 01 diseñamos una herramienta que clasifica reseñas de una tienda de café (ficticia) y escribe un informe mensual. Ahí aparecieron cuatro defensas: lista cerrada, set dorado, filtros y números calculados por fuera de la IA. Las vimos rápido, porque el objetivo era el diseño completo.

Esta guía se mete a fondo en esas cuatro defensas. Es la diferencia entre una herramienta que **parece** que funciona y una que puedes defender cuando alguien te pregunta «¿y eso cómo lo sabes?».

Si no leíste la guía 01, no pasa nada: aquí se explica todo de nuevo con el mismo ejemplo.

Las cuatro maneras en que una IA «inventa»

Cuando la gente dice que la IA «alucina», casi siempre está hablando de una de estas cuatro cosas. Cada una tiene su defensa.

Cómo inventa	Cómo se ve	La defensa	Dónde está
Categorías	«Envíos», «entrega», «logística»: tres nombres para lo mismo	Lista cerrada con definiciones	Parte 1
Formato	El JSON llega cortado o con campos de más	Esquema, validación y reintento	Parte 2
Cifras y citas	«El 40 % se queja del envío» cuando era 31 %; una frase que el cliente nunca escribió	Números calculados por fuera y verificación frase por frase	Parte 3
Deriva	Funciona en la prueba y se desvía a la reseña doscientas	Set dorado, filtros que bloquean y registro de bloqueos	Partes 4 y 5

Qué te llevas

- Una plantilla para definir categorías que la IA no confunda.
- Un esquema de salida y un validador con reintento, en Python.
- Un verificador de cifras y uno de citas textuales, listos para adaptar.
- Cómo armar, medir y mantener un set dorado, con una matriz de confusión leída paso a paso.
- Seis familias de filtros, con código para las que más se usan.

> **Ojo:** el código está en Python porque es el más fácil de leer, pero cada bloque va explicado en palabras. Si no programas, la lógica te sirve igual para pedírsela a la IA o para montarla en una hoja de cálculo.

Parte 1 • Diseña la lista cerrada

Una lista cerrada es la lista de valores que la IA puede escoger. Parece lo más fácil de la herramienta y es donde más se equivoca la gente, porque la arma en cinco minutos, de memoria, y después todos los conteos dependen de ella.

Cómo nace una buena lista

1. **Lee 100 casos reales a mano.** No inventes las categorías desde el escritorio: salen de los datos.
2. **Ponle a cada uno una etiqueta libre**, la que te salga: «el café llegó molido y lo pedí en grano», «me cobraron dos veces», «el domiciliario fue grosero».
3. **Agrupa las etiquetas libres** que llevan a la misma acción. «Me cobraron dos veces» y «no me llegó la factura» pueden ir juntas en `cobro` si las resuelve la misma persona.
4. **Ponle nombre a cada grupo** y escribe su definición (abajo está la plantilla).
5. **Cuenta cuántos casos quedaron sin grupo.** Esos van a `no_mapeado`. Si son muchos, te faltan categorías; si son poquitos y muy raros, está bien que se queden ahí.

Tres reglas para que funcione

- **Tantas categorías como decisiones distintas tomas.** Si dos categorías llevan a la misma acción, júntalas. Si una categoría tiene casos que van a personas distintas, pártela. La lista no es para describir el mundo: es para decidir qué hacer.
- **Que no se pisen.** Cada caso debería caer en una sola categoría. Cuando no se puede (una reseña que se queja del envío y de la atención), escribe la regla de desempate y ponla en la definición.
- `no_mapeado` **siempre existe.** Sin esa salida, la IA mete a la fuerza cada caso raro en la categoría que más se le parece, y eso no se ve en ningún lado.

La definición es más importante que el nombre

El cambio que más mejora una clasificación no es cambiar de modelo: es pasar de darle a la IA **nombres** («envío, atención») a darle **definiciones con fronteras**. La IA no sabe que para la tienda «el pedido no llega y nadie contesta» es un problema de envío y no de atención; eso lo decides tú y lo escribes.

PLANTILLA · FICHA DE CATEGORÍA

Campo	Qué va
Nombre	En minúsculas, sin tildes ni espacios: `envio`
Definición	Una frase: qué es
Incluye	Los casos que sí van aquí, en concreto
No incluye	Los casos que se parecen pero van en otra (y en cuál)
Ejemplo claro	Un caso real que nadie discutiría
Ejemplo frontera	Un caso dudoso y por qué se decidió así

Ejemplo: la frontera entre envío y atención

Campo	envio	atencion
Definición	Problemas o elogios sobre la entrega del pedido	Problemas o elogios sobre el trato y las respuestas del equipo
Incluye	Demora, pedido perdido, llegó dañado por el transporte, domiciliario	Respuestas por chat o correo, tono, tiempos de respuesta, solución de reclamos
No incluye	El empaque roto por mal empacado (va en `empaque`)	Un pedido que no llega aunque mencione que nadie contesta (va en `envio`)
Ejemplo claro	«Pedí el lunes y llegó el viernes»	«Me respondieron rapidísimo y me cambiaron la bolsa»
Ejemplo frontera	«Llevo diez días esperando y nadie me responde»: **envio**, porque lo que el cliente quiere es su café	«El domiciliario me dijo que llamara a la tienda y nunca me contestaron»: **atencion**, porque el pedido sí llegó

Esa última columna vale oro. Los ejemplos frontera son los que más se equivoca la IA, y también las personas. Si los dejas escritos, la IA y tu equipo deciden igual.

El campo de descubrimiento

Una lista cerrada tiene un riesgo: te vuelve ciego a lo nuevo. Si el mes que viene la gente empieza a quejarse de la app de pedidos y tú no tienes esa categoría, todo cae en `no_mapeado` y nadie lo nota.

La solución es pedir **dos cosas separadas** en la misma respuesta:

- `tema`: cerrado, de la lista. **Este es el que se cuenta.**
- `tema_sugerido`: libre, opcional. Solo si el caso no encaja bien en la lista, la IA propone cómo lo llamaría.

Así los conteos nunca se rompen, y cada mes revisas los `tema_sugerido` para ver si toca abrir una categoría nueva.

Cuándo revisar la lista

- Si `no_mapeado` crece de un mes a otro, o pasa del umbral que decidiste (yo arrancaría con un 10 % y lo ajustaría con la experiencia; es un criterio, no una ley).
- Si un mismo `tema_sugerido` aparece varias veces.
- Si cambia el negocio: un producto nuevo, un canal de venta nuevo, una política nueva.

Versiona la lista

Cada vez que la lista cambia, cambia su versión (`v1`, `v2`…) y cada resultado guarda con qué versión se clasificó. Si no, en tres meses vas a comparar meses que se clasificaron con listas distintas sin saberlo.

v1	v2	Qué hacer con lo clasificado en v1
envio	envio	Nada
atencion	atencion	Nada
no_mapeado	app (nueva)	Reclasificar solo los `no_mapeado` de v1
producto	producto_sabor, producto_molienda	Reclasificar solo los de `producto`

Fíjate en la última columna: **no se reclasifica todo**, solo lo que el cambio afecta. Eso ahorra plata y evita mover cifras que no tenían por qué moverse.

Parte 2 • Haz la salida verificable

Una salida verificable es una que un programa (o una hoja) puede revisar sin leerla como persona. Eso se consigue con tres cosas: un esquema, evidencia y confianza.

El esquema

El esquema es la descripción exacta de lo que debe llegar: qué campos, de qué tipo, con qué valores permitidos, cuáles son obligatorios y que no sobre nada. El formato estándar para escribirlo se llama JSON Schema.

```
{
  "type": "object",
  "required": ["evidencia", "tema", "sentimiento", "confianza"],
  "additionalProperties": false,
  "properties": {
    "evidencia":    {"type": "string", "maxLength": 200},
    "tema":         {"enum": ["producto", "envio", "cobro",
                              "atencion", "empaque", "no_mapeado"]},
    "sentimiento":  {"enum": ["positivo", "neutro", "negativo"]},
    "confianza":    {"enum": ["alta", "baja"]},
    "tema_sugerido": {"type": "string", "maxLength": 40}
  }
}
```

Tres detalles que importan:

- `additionalProperties: false`**:** si la IA agrega un campo que no pediste, la respuesta no pasa. Parece exagerado, pero un campo de más suele ser la señal de que la IA no entendió el formato.
- `evidencia` **va primero.** Pedir la cita antes que la etiqueta hace que la IA «mire» el texto antes de decidir. En mi experiencia las etiquetas salen más consistentes así, y además te queda con qué revisar.
- `maxLength`**:** pone techo a lo que se puede escribir. Una evidencia de 2.000 caracteres no es una evidencia.

> **Salida estructurada nativa:** varios proveedores de IA tienen un modo que obliga a la respuesta a cumplir un esquema. Si el tuyo lo tiene, úsalo. Pero **valida igual**: el modo nativo garantiza la forma, no que la cita sea textual ni que la etiqueta tenga sentido.

El validador

Esta función recibe el texto que devolvió la IA y responde dos cosas: el resultado y la lista de problemas. Si la lista está vacía, pasa.

```
import json

PERMITIDOS = {
    "tema": {"producto", "envio", "cobro", "atencion", "empaque", "no_mapeado"},
    "sentimiento": {"positivo", "neutro", "negativo"},
    "confianza": {"alta", "baja"},
}
OBLIGATORIOS = {"evidencia", "tema", "sentimiento", "confianza"}
OPCIONALES = {"tema_sugerido"}

def validar(texto):
    try:
        r = json.loads(texto)
    except json.JSONDecodeError:
        return None, ["no es JSON válido (¿se cortó la respuesta?)"]
    problemas = []
    faltan = OBLIGATORIOS - r.keys()
    sobran = r.keys() - OBLIGATORIOS - OPCIONALES
    if faltan:
        problemas.append(f"faltan campos: {sorted(faltan)}")
    if sobran:
        problemas.append(f"sobran campos: {sorted(sobran)}")
    for campo, valores in PERMITIDOS.items():
        if campo in r and r[campo] not in valores:
            problemas.append(f"{campo} = {r[campo]!r} no está en la lista")
    if len(r.get("evidencia", "")) > 200:
        problemas.append("evidencia demasiado larga")
    return r, problemas
```

El reintento con el error a la vista

Cuando la respuesta no pasa, no la arregles tú «a ojo» ni adivines lo que faltó. Devuélvele a la IA **el problema concreto** y pídele que corrija. Una vez. Si vuelve a fallar, el caso va a revisión humana.

```
def clasificar(resena, llamar_ia, intentos=2):
    prompt = armar_prompt(resena)
    for _ in range(intentos):
        respuesta = llamar_ia(prompt, max_tokens=1000)
        r, problemas = validar(respuesta)
        if not problemas:
            return r
        prompt = (armar_prompt(resena)
                  + "\n\nTu respuesta anterior tuvo estos problemas: "
                  + "; ".join(problemas)
                  + ". Responde de nuevo SOLO con el JSON corregido.")
    return {"estado": "revision", "problemas": problemas}
```

`llamar_ia` y `armar_prompt` son las funciones que conectan con tu proveedor y arman el texto; cambian según la herramienta que uses, la lógica no.

El presupuesto de respuesta

`max_tokens` es el límite de lo que la IA puede escribir. Si lo dejas justo, el JSON llega cortado a la mitad y el validador lo rechaza: pierdes la llamada. Dale varias veces lo que esperas. Para una respuesta como esta, de unas 80 palabras, 1.000 tokens sobran. En salidas JSON grandes (muchas etiquetas por caso, o descripciones de imágenes) he tenido que subir a 5.000 o 6.000.

Parte 3 • Cifras y citas que no se inventan

Esta es la parte que más vergüenza ahorra. Un informe con una cifra mal o con una cita que el cliente nunca escribió se cae en la primera pregunta, y con él se cae la confianza en todo lo demás.

La regla: la IA no cuenta

La IA es buena leyendo y redactando, y mala contando. Así que **no le pidas que cuente**. Los números los calcula una fórmula o un programa, y la IA recibe una «hoja de datos» ya hecha para redactar sobre ella.

```
Redacta el informe del mes con SOLO estos datos.
No calcules nada nuevo, no redondees distinto y no agregues cifras.
Si algo no está en los datos, no lo digas.

DATOS (octubre):
total_resenas: 300
envio: 93  (31 %)
producto: 81  (27 %)
atencion: 57  (19 %)
empaque: 39  (13 %)
cobro: 30  (10 %)
urgencia_alta: 24
frases_clave_envio: [...]
```

Fíjate en que hasta los porcentajes van calculados. Si le das 93 y 300, la IA puede «calcular» un 30 % o un 32 %; si le das 31 %, solo tiene que copiar.

El verificador de cifras

Aun con la hoja de datos, la IA a veces redondea, mezcla o se inventa un número. Entonces, antes de que el informe salga, un programa saca **todas las cifras del texto** y revisa que cada una exista en los datos.

```
import re

def cifras_del_texto(texto):
    # Formato colombiano: 1.234 son miles y 12,5 es decimal
    encontradas = re.findall(r"\d+(?:[.,]\d+)*", texto)
    return [float(c.replace(".", "").replace(",", ".")) for c in encontradas]

def verificar_cifras(informe, datos, permitidas_extra=()):
    validas = {round(float(v), 1) for v in datos.values()} | set(permitidas_extra)
    return [c for c in cifras_del_texto(informe) if round(c, 1) not in validas]
```

Los `datos` son la misma hoja de datos, pero como diccionario y con los porcentajes ya incluidos:

```
datos = {"total_resenas": 300, "envio": 93, "pct_envio": 31,
         "producto": 81, "pct_producto": 27, "urgencia_alta": 24}
```

Si `verificar_cifras` devuelve algo, el informe no sale. `permitidas_extra` es para números que sí pueden aparecer y no están en los datos, como el año o el número de días del mes.

El verificador de citas

La misma idea para las citas: cada frase entre comillas del informe, y cada `evidencia` de la clasificación, tiene que estar **de verdad** en la reseña. El truco es comparar textos «normalizados»: sin mayúsculas, sin tildes, sin comillas ni signos, para que una diferencia de puntuación no tumbe una cita buena.

```
import re, unicodedata

def normalizar(t):
    t = unicodedata.normalize("NFKD", t.lower())
    t = "".join(c for c in t if not unicodedata.combining(c))
    t = re.sub(r"[«»\"'“”‘’.,;:!¡?¿()]", " ", t)
    return " ".join(t.split())

def cita_es_textual(cita, original):
    return normalizar(cita) in normalizar(original)
```

Es un filtro de diez líneas que pilla una de las formas más comunes de inventar: la IA «resume» la queja con sus palabras y la presenta como si fuera del cliente.

> **Las citas que se publican se leen a mano.** El verificador garantiza que la frase existe; no garantiza que se entienda igual fuera de contexto. Una cita que se va a mostrar a un cliente o a un jefe la lee una persona y decide si su tono es inequívoco.

Guarda cada respuesta por su huella

Si mandas exactamente el mismo texto al mismo modelo dos veces, no tiene sentido pagar dos veces ni arriesgarte a recibir dos respuestas distintas. La solución es guardar cada respuesta con una **huella** del texto que la produjo: un código único que sale del contenido (se llama *hash*).

```
import hashlib, json, pathlib

CACHE = pathlib.Path("cache")
CACHE.mkdir(exist_ok=True)

def con_cache(prompt, modelo, llamar_ia):
    huella = hashlib.sha256(f"{modelo}\n{prompt}".encode()).hexdigest()
    archivo = CACHE / f"{huella}.json"
    if archivo.exists():
        return json.loads(archivo.read_text())
    respuesta = llamar_ia(prompt)
    archivo.write_text(json.dumps(respuesta, ensure_ascii=False))
    return respuesta
```

Con esto ganas tres cosas: no vuelves a pagar lo que ya pagaste, puedes reproducir exactamente el informe de cualquier mes y, si alguien pregunta «¿por qué dijo esto?», tienes la respuesta original guardada.

Parte 4 • El set dorado a fondo

El set dorado es tu examen: casos etiquetados a mano con cuidado contra los que comparas lo que hace la herramienta. En la guía 01 armamos uno de 50 casos. Aquí va todo lo que hace que ese examen sea confiable.

Cuántos casos y cuáles

El tamaño no se decide en total: se decide **por categoría**. La razón es una cuenta sencilla:

Casos de una categoría en el set	Cuánto mueve la cobertura un solo error
10	10 puntos
20	5 puntos
50	2 puntos

Con 10 casos de `cobro`, un error baja la cobertura del 100 % al 90 %. No sabes si la herramienta es mala o si tuviste mala suerte con un caso. Por eso, para las categorías que importan, apunta a **20 casos o más**. Las que casi no aparecen pueden tener menos, sabiendo que su cifra es más ruidosa.

Escoge los casos al azar dentro de cada categoría, e incluye a propósito algunos casos frontera. Un examen con solo casos fáciles da notas que no sirven.

Dos personas, por separado

Si puedes, que dos personas etiqueten el set sin ver lo que hizo la otra. Después cuentas en cuántos casos coincidieron.

- Si coinciden en 86 de 100, el **acuerdo** es del 86 %.
- Lo importante no es la cifra, es **dónde no coinciden**. Si casi todos los desacuerdos están entre `envio` y `atencion`, el problema no es de las personas: es de la definición. Se ajusta la ficha de categoría y se vuelve a etiquetar.

Hay medidas más finas que descuentan las coincidencias por azar (la más conocida es la kappa de Cohen), pero para empezar el porcentaje de acuerdo y la lista de desacuerdos te dicen casi todo.

> **Una idea incómoda:** si dos personas de tu equipo solo coinciden en el 70 % de los casos, no le pidas a la IA un 95 %. La IA no puede ser más consistente que la definición que le diste.

Tres sets, no uno

Este es el error que más me ha costado. Mientras mejoras el prompt, lo pruebas contra el set dorado, ajustas, vuelves a probar… y sin darte cuenta lo estás ajustando **para ese examen**. Me pasó con un buscador que armé: quedó en 100 % sobre los casos con los que lo ajusté y en 80 % sobre casos que no había visto. Eso se llama sobreajuste.

La solución es tener tres sets:

Set	Para qué	Regla
Ajuste	Mejorar el prompt y las definiciones	Lo miras todo lo que quieras
Control	Medir cada cambio	No lo usas para ajustar; solo para medir
Ciego	La prueba final antes de usar la herramienta en serio	Se escribe **después** de terminar de ajustar y se corre **una sola vez**

Y una regla para cuando algo falla: **corrige la causa general, nunca el caso**. Si la herramienta confunde «domicilio» con «dirección», no agregues al prompt «la reseña 47 es de envío»: agrega a la definición que las quejas sobre la dirección de entrega son de `envio`. Lo primero arregla un caso; lo segundo, todos los parecidos.

La matriz de confusión

La matriz de confusión es una tabla que muestra, para cada categoría real, a dónde la mandó la herramienta. Es la mejor forma de ver **qué** confunde, no solo cuánto se equivoca. Ejemplo con un set de 100 reseñas:

Real / Herramienta	producto	envio	cobro	atencion	empaque	Total
producto	**28**	0	0	1	1	30
envio	0	**15**	0	5	0	20
cobro	0	0	**9**	1	0	10
atencion	1	2	0	**22**	0	25
empaque	1	0	0	0	**14**	15
Total	30	17	9	29	15	100

La diagonal (en negrita) son los aciertos: 88 de 100. Pero lo útil está fuera de la diagonal: **5 reseñas de envío se fueron a atención**. Es exactamente el caso frontera de la Parte 1. La solución no es «un mejor modelo»: es escribir el ejemplo frontera en la definición.

Las cifras por categoría

Categoría	Precisión	Cobertura	F1
producto	28 ÷ 30 = 93 %	28 ÷ 30 = 93 %	0,93
envio	15 ÷ 17 = 88 %	15 ÷ 20 = 75 %	0,81
cobro	9 ÷ 9 = 100 %	9 ÷ 10 = 90 %	0,95
atencion	22 ÷ 29 = 76 %	22 ÷ 25 = 88 %	0,81
empaque	14 ÷ 15 = 93 %	14 ÷ 15 = 93 %	0,93
Promedio (F1 macro)			**0,89**

Recordatorio rápido: **precisión** es «de lo que marcó como X, cuánto era X»; **cobertura** es «de todo lo que era X, cuánto encontró». El F1 junta las dos en una sola cifra. El **F1 macro** es el promedio simple de las categorías, para que una categoría grande y fácil no esconda a una pequeña y difícil.

Lectura del ejemplo: `envio` encuentra pocas (75 % de cobertura) y `atencion` se lleva casos ajenos (76 % de precisión). Son las dos caras del mismo problema. Arreglas la frontera y suben las dos.

Mantener el set vivo

- Agrega casos nuevos cada trimestre, sobre todo de lo que más cambió.
- Si cambias la lista de categorías, el set dorado se reetiqueta con la versión nueva.
- Vuelve a medir cada vez que cambies de modelo, de prompt o de lista. Un modelo nuevo «mejor» puede ser peor en tu tarea.

Parte 5 • Filtros a fondo: candados antes, auditor después

En la guía 01 vimos la regla: un filtro bloquea, no avisa. Aquí va cómo se organiza un sistema de filtros completo. Hay dos momentos para poner defensas.

Candados: antes de que la IA responda

Los candados van en lo que le das a la IA. Reducen las oportunidades de inventar:

- **Inventario exacto:** en vez de «redacta el informe», «redacta el informe usando solo estos datos y nada más».
- **Campos literales:** lo que tiene que aparecer tal cual (un nombre de producto, una cifra) va como dato aparte, no mezclado en las instrucciones.
- **Lista cerrada en la entrada:** si la IA tiene que escoger, que escoja de una lista que está en el mismo prompt.
- **Pocas cosas por llamada:** si le das 50 reseñas en una sola llamada, empieza a mezclar unas con otras. Una por llamada, o pocas y bien separadas.

Ojo con un detalle: **las palabras pesan más de lo que crees**. Si en la descripción de una categoría escribes un ejemplo muy llamativo, la IA tiende a ver ese ejemplo en todas partes. Los ejemplos de las definiciones tienen que ser variados.

Auditor: después de que la IA responde

El auditor revisa la salida y decide si sale o no. Estas son las seis familias de filtros que uso:

Familia	Qué revisa	Ejemplo en la tienda
Forma	Que cumpla el esquema	El validador de la Parte 2
Lista	Que los valores sean de la lista cerrada	`tema` dentro de los seis permitidos
Fidelidad	Que cifras y citas existan en los datos	Los verificadores de la Parte 3
Coherencia	Que los campos no se contradigan entre sí	«Positivo» con 1 estrella no pasa
Vetados y muletillas	Palabras prohibidas y frases genéricas que no dicen nada	«Experiencia única», «en general los clientes están satisfechos»
Datos personales	Que no se cuelen correos, teléfonos o nombres	Un correo copiado de la reseña dentro del informe

Código para vetados y datos personales

```
import re

MULETILLAS = ["experiencia única", "de alta calidad", "en general",
              "cabe destacar", "sin duda", "los clientes están satisfechos"]
CORREO = re.compile(r"[\w.+-]+@[\w-]+\.[\w.]+")
CELULAR_CO = re.compile(r"(?:\+?57[\s-]?)?3\d{2}[\s-]?\d{3}[\s-]?\d{4}")

def auditar_texto(texto):
    problemas = []
    bajo = texto.lower()
    for m in MULETILLAS:
        if m in bajo:
            problemas.append(f"muletilla genérica: «{m}»")
    if CORREO.search(texto):
        problemas.append("hay un correo electrónico")
    if CELULAR_CO.search(texto):
        problemas.append("hay un número de celular")
    return problemas   # si no está vacía, el texto NO sale
```

Las muletillas son un filtro de calidad muy subestimado. Una frase como «en general los clientes están satisfechos» no es falsa: es vacía, y un informe lleno de frases vacías es un informe que nadie lee. Bloquearlas obliga a que el texto diga algo concreto.

El crítico: una IA que revisa a la otra

Hay cosas que un filtro con reglas no ve: un informe que es técnicamente correcto pero saca una conclusión que los datos no sostienen, o que esconde lo importante en el último párrafo. Para eso sirve un **crítico**: un segundo llamado a la IA, con un rol y una lista de revisión, que lee la salida y dice si es publicable.

```
Eres el revisor del informe mensual de reseñas. No lo reescribas.
Revisa y responde en JSON con "publicable" (sí/no) y "problemas" (lista):
1. ¿Cada conclusión se sostiene con los datos dados? Cita cuál no.
2. ¿Lo más grave (urgencia alta) aparece en el primer párrafo?
3. ¿Hay alguna cifra sin su denominador (un % sin decir de qué total)?
4. ¿Hay recomendaciones que no salen de los datos?
```

Tres reglas para que el crítico sirva:

- **No es el único filtro.** Va después de los filtros con reglas, nunca en su lugar. Las reglas no se cansan ni cambian de opinión.
- **Al crítico también se le mide.** Pásale informes buenos y malos que tú ya calificaste y mira si acierta. Un crítico que aprueba todo es peor que no tener crítico.
- **Corre después de cada cambio.** Cada vez que cambias una regla, un prompt o el modelo, el crítico revisa una corrida completa antes de darla por buena.

Registra lo que se bloquea

Cada bloqueo se guarda con su motivo. Una vez por semana miras cuántos hubo y de qué tipo. Esa cifra te avisa antes que nadie:

- Si los bloqueos de **forma** se disparan, probablemente cambió algo del modelo o del proveedor.
- Si suben los de **lista**, están llegando casos que tu lista no cubre.
- Si suben los de **fidelidad**, la IA está «resumiendo» de más: revisa el prompt de redacción.

Todo junto: una corrida de la tienda

Así se ve un mes de la herramienta completa, con las defensas de esta guía. Los números son de ejemplo.

Paso	Qué pasa	Resultado
1	Llegan las reseñas y se les quitan los datos personales	300 reseñas limpias
2	Primera pasada de clasificación, con validador y reintento	297 válidas; 3 a revisión por formato
3	Filtros de lista, fidelidad de la evidencia y coherencia	290 pasan; 4 citas no textuales y 3 incoherencias a revisión
4	Segunda pasada para las de confianza baja	41 dudosas: 34 se resuelven, 7 a revisión
5	Revisión humana	17 casos, unos 20 minutos
6	Conteos calculados por programa	Hoja de datos del mes
7	Redacción del informe sobre la hoja de datos	Borrador
8	Verificador de cifras y de citas, auditor de muletillas y datos personales	1 cifra mal redondeada: se corrige y se regenera
9	Crítico	Publicable; sugiere subir el tema de envío al primer párrafo
10	Aprobación de una persona	Informe enviado

Fíjate en el paso 5: la herramienta no elimina el trabajo humano, lo **concentra** en los 17 casos donde de verdad hace falta criterio, en vez de repartirlo en 300 lecturas.

Errores que ya vi

Error	Cómo se ve	Qué hacer
Lista hecha de memoria	Muchos `no_mapeado` o categorías que nadie usa	Que la lista salga de leer 100 casos reales
Dar nombres sin definiciones	Envío y atención se confunden siempre	Ficha de categoría con ejemplo frontera
Sin campo de descubrimiento	Un problema nuevo pasa meses sin verse	`tema_sugerido` libre, revisado cada mes
Cambiar la lista sin versionar	Meses que no se pueden comparar	Versión de la lista guardada en cada resultado
Arreglar el JSON a mano	Datos que nadie sabe de dónde salieron	Validador + reintento con el error a la vista
Pedirle a la IA que cuente	Cifras que no cuadran	Hoja de datos calculada + verificador de cifras
Confiar en que la cita es textual	Frases que el cliente nunca escribió	Verificador de citas + lectura humana de lo que se publica
Ajustar contra el examen	100 % en pruebas, mucho menos en la vida real	Tres sets: ajuste, control y ciego
Corregir el caso, no la causa	El mismo error vuelve con otro caso	Arreglar la definición, no agregar excepciones
Crítico sin medir	Aprueba todo	Medirlo con informes buenos y malos conocidos

Lista de verificación

- ☐ La lista salió de leer al menos 100 casos reales.
- ☐ Cada categoría tiene ficha: definición, incluye, no incluye, ejemplo claro y ejemplo frontera.
- ☐ Existe `no_mapeado` y sé a partir de qué cifra reviso la lista.
- ☐ Hay un campo libre de descubrimiento separado del campo que se cuenta.
- ☐ La lista tiene versión y cada resultado guarda con cuál se clasificó.
- ☐ Tengo un esquema con campos obligatorios, valores permitidos y nada de más.

- ☐ La evidencia se pide antes que la etiqueta.
- ☐ Un validador revisa cada respuesta y reintenta una vez con el error a la vista.
- ☐ El límite de respuesta es holgado.
- ☐ La IA no cuenta: recibe una hoja de datos calculada.
- ☐ Un verificador revisa cada cifra del texto contra los datos.
- ☐ Un verificador revisa que cada cita sea textual, y las que se publican las lee una persona.

- ☐ Las respuestas se guardan por huella y no se vuelven a pagar.
- ☐ El set dorado tiene 20 casos o más en las categorías que importan.
- ☐ Sé dónde no coinciden las personas que etiquetaron.
- ☐ Tengo set de ajuste, de control y ciego, y el ciego se corrió una sola vez.
- ☐ Leí la matriz de confusión y arreglé causas, no casos.
- ☐ Los filtros cubren las seis familias que aplican a mi caso.

- ☐ El crítico va después de los filtros y está medido.
- ☐ Los bloqueos se registran y los reviso cada semana.

De dónde sale esto

Todo lo de esta guía lo he aplicado en herramientas reales de estrategia y marketing: clasificadores de conversación, etiquetado de imágenes con listas cerradas, buscadores sobre bases de conocimiento y motores que redactan informes. El ejemplo de la tienda de café y todos sus números son **ficticios**, para que puedas ver el método sin datos de nadie.

El umbral del 10 % para revisar la lista y los 20 casos por categoría son criterios míos, con la cuenta que los sostiene; ajústalos a lo que esté en juego en tu caso.

No soy programador; vengo de la estrategia. Si alguien de ingeniería haría algo de esto distinto, me sirve que me lo diga.

En la próxima guía: agentes y automatización. Cómo hacer que todo esto corra solo, sin que nadie lo tenga que lanzar desde un chat, y sin perder el control.

Si esta guía te sirvió, guárdala y pásasela a alguien que esté peleando con una IA que se inventa cosas.